2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP) | 979-8-3503-7647-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/INNOCOMP63224.2024.00014

Automatic Devanagari Text Summarization for Youtube Videos

Sanika Patil
Department Of Information Technology
Sardar Patel Institute Of Technology
Mumbai, India
sanika.patil@spit.ac.in

Shraddha Nandvikar
Department Of Information Technology
Sardar Patel Institute Of Technology
Mumbai, India
shraddha.nandvikar@spit.ac.in

Aakash Pardeshi
Department Of Information Technology
Sardar Patel Institute Of Technology
Mumbai, India
aakash.pardeshi@spit.ac.in

Prof. Swapnali Kurhade
Department Of Information Technology
Sardar Patel Institute Of Technology
Mumbai, India
swapnali.kurhade@spit.ac.in

Abstract – Devanagari, increasingly used as a major language of internet communication, is also India's official language and widely spoken worldwide. This research focuses on summarization in Devanagari. Text summarization is becoming vital due to the abundance of information available. It aids in understanding documents efficiently, streamlining articles into concise versions. Both extractive and abstractive approaches are used to summarize texts, with extractive methods ranking sentences based on importance and abstractive methods deriving natural language representations. This work aims to comprehensively examine text summarization techniques, emphasizing coherence and reducing redundancy, particularly in Devanagari.

Keywords – Automatic Text Summarization, NLP, Video Summarization, T5, BART, LexRank, abstractive summarization, extractive summarization.

I. Introduction

In today's internet age, we're flooded with tons of information every day. Sorting through all this content, especially online news and articles, can be overwhelming. Summarizing all this text manually takes a lot of time and sometimes isn't very accurate. That's why researchers are working on ways to use computers to summarize text quickly and accurately. While there are many tools for summarizing English text, languages like Devanagari and Marathi don't have as many options. Our project focuses on creating a special summarization tool for Devanagari and Marathi languages using a smart computer method. We're mainly looking at a method where the computer picks out the most important words and sentences to make a summary. Our goal is to help people quickly understand video content in Devanagari by summarizing it into easy-to-read text. It's all about making summarization better for Indian languages.

After running the text extracted from the video through a summarizer that uses language models that have already been trained from the Hugging Face library, a summary is produced. In general, summarizers can be classified as either extractive (traditional) or abstractive (advanced). The Facebook BART Transformer model [7] (Abstractive) is a machine learning approach that combines the BART and Transformer algorithms. Furthermore, the two algorithms are fused to take advantage of their complementary features. BART is a regression-focused model that combines a number of traditional approaches. Transformer, on the other hand, is a recurrent model that doesn't employ parameter optimization. Using encoders and decoders, it employs a transformer-based methodology to provide an abstractive synopsis of the extracted text. Additionally, other noteworthy summarization methods include LexRank[9], which employs graph-based techniques, TF-IDF, a statistical measure for word importance, and Whisper[5] by OpenAI, a model known for generating abstractive summaries by leveraging a neural network framework.And T5[10], which is like a super flexible tool that's good at lots of language tasks, including making summaries by writing new sentences. These different methods help turn big chunks of text into short, understandable summaries.
These diverse techniques contribute to the landscape of text summarization, each offering unique strengths in distilling information into concise synopsis.

Extractive Summarization:

This involves identifying and pulling out essential sentences or phrases directly from the source text. [12] These extracted elements are selected based on predefined measures like word frequency, importance, relevance, or other statistical methods.
How it works: The process typically includes:

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:05:08 UTC from IEEE Xplore. Restrictions apply.

- *Text Preprocessing:* Breaking down the text into sentences, tokenization, and identifying important words or phrases.
- *Scoring or Ranking:* Each sentence or phrase is assessed based on relevance or significance using metrics like TF-IDF (Term Frequency-Inverse Document Frequency), sentence position, or other algorithms.
- *Selection:* Sentences or phrases with the highest scores are chosen to form the summary, typically without any modifications.

Encoder-Decoder Role: In the context of extractive summarization[15], the encoder can represent the original text by converting it into a numerical or vectorized format, capturing the information present in the text. The decoder, in this case, might not be explicitly utilized, as extractive methods often do not generate entirely new sentences but rather select and reorganize existing content.

Abstractive Summarization:

This involves generating entirely new sentences that may not exist in the source text[13], effectively rephrasing and reorganizing information to form a concise summary.
How it works: The process typically includes:

- *Text Understanding:* The model must comprehend the meaning and context of the original text, often using deep learning architectures like neural networks.
- *Content Generation:* The model generates new sentences by predicting or generating words, phrases, or sentences based on its understanding of the input text.
- *Language Generation:* It focuses on maintaining coherence, grammar, and context while generating these new sentences.

Encoder-Decoder Role: In abstractive summarization[14], the encoder converts the input text into a latent representation, capturing its semantics. The decoder then uses this representation to generate new content, rephrasing and restructuring the information to create a summary while preserving the essence of the original text.

II. Literature Survey

Shubhankita Sood, M. Ferni Ukrit and Rishy Parasar [3] delve into foundational works in text summarization exploring traditional and modern techniques introduced by LexRank . This paper contributes by empirically comparing extractive techniques across Devanagari, English, and French, enhancing understanding through ROUGE-based assessments. The exploration of the underdeveloped domain of ATS [8] in Devanagari. Introducing a machine learning model with notable metrics (F-Score 58%, Rouge Score 67.5%), it aligns with [3], which empirically compares extractive techniques across languages.

R. Karmakar, K. Nirantar, P. Kurunkar, P. Hiremath and D. Chaudhari [4] focuses on LSTM-based abstractive text summarization, emphasizing sequence-to-sequence modeling for Indian languages. It stresses the scarcity of suitable datasets and introduces newly generated ones for this purpose. Addressing challenges like refining stop words and handling linguistic nuances, the survey outlines future directions involving data augmentation, web scraping, API development, and the integration of advanced modeling techniques for enhanced summarization capabilities.

Nandidni Kapoor and kapil gupta [1] go over a study that applies the Real Coded Genetic Algorithm to summarize Devanagari Movie reviews. It achieves a 65% reduction in summary size, emphasizing significant sentences through theme analysis. also recently extractive and abstractive summarization techniques for Devanagari text, employing hierarchical agglomerative clustering [11] and the PageRank [2] algorithm. It innovatively proposes a POS tagger-based multi-sentence compression approach for efficient abstractive summarization in Devanagari.

S. Degadwala, D. Vyas, K. N. Patel, M. Soni, P. Parkash Singh and R. Maranan [2] A novel PageRank-based approach for optimizing Devanagari paragraph summarization was described by Sheshang Dagadwala and colleagues. It leverages inherent paragraph structure and linguistic features to identify central sentences, outperforming existing methods in informativeness and coherence.

Challenges specific to Devanagari text summarization

- One of the primary challenges in Devanagari text summarization is the limited availability of high-quality datasets. Existing datasets may be small in size, lack diversity in topics and genres, or have inadequate annotations for summarization tasks.

- Devanagari script includes punctuation marks such as exclamation points, semicolons, and colons that differ from those in Latin scripts. Existing models may struggle to effectively process these punctuation marks, leading to errors in summarization output.

- Stop words in Devanagari languages may differ from those in English or other widely studied languages. Refining the list of stop words specific to Devanagari languages is essential to improve the quality of generated summaries.

III. Proposed Methodology

The process begins with prompting the user to paste a YouTube URL. If the video linked has adequate subtitles, the system will generate both abstractive and extractive summarizations of the content. However, if subtitles are not available, the system will resort to using Whisper AI to generate a transcript. Once the Devanagari transcript text is obtained, it will undergo summarization either through abstractive or extractive methods discussed below.

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:05:08 UTC from IEEE Xplore. Restrictions apply.

A. Data Collection

Audio Transcription and Text Extraction:
The research began by extracting textual content from a relevant Devanagari YouTube video using the `pytube` library. The `whisper` library was employed to transcribe the audio content, yielding a comprehensive dataset of Devanagari text for subsequent analysis.

The Whisper automatic speech recognition (ASR) system, trained on a vast and diverse dataset, was employed to transcribe audio content from various sources [5]. Its extensive training enhances robustness, allowing accurate transcription despite accents, background noise, and technical language variations.

Codebase Dependencies:
The research leverages essential Python packages, notably HuggingFace Transformers for efficient text tokenization and ffmpeg-python for seamless audio file processing during preprocessing.

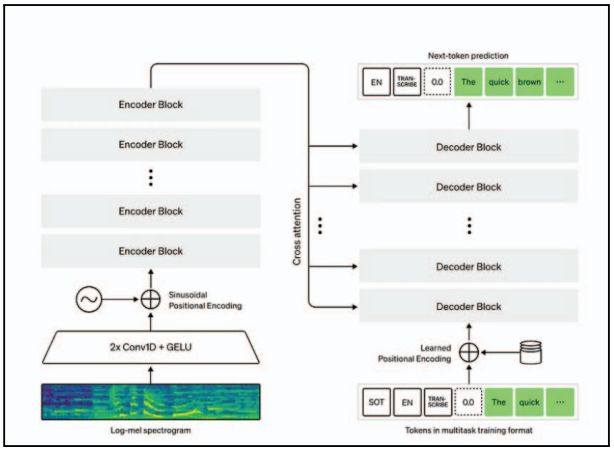

Fig 1 : Whisper Architecture

The encoder-decoder Transformer [5] is the implementation of the straightforward end-to-end Whisper architecture. The audio input is divided into 30-second segments, transformed into a log-Mel spectrum, and subsequently fed into an encoder. With the help of unique tokens that instruct the single model to carry out tasks like multilingual speech transcription, language identification, phrase-level timestamps, and to-English speech translation, a decoder is trained to anticipate the matching text caption.

Also OpenAI's Whisper ASR used for Portuguese speech recognition, it showcases [6] its potential to address punctuation challenges in prior models. certain punctuation marks like exclamation, semicolon, and colon may benefit from improvements.

The Whisper ASR system, coupled with Python packages, forms the foundation for feature extraction. By transcribing audio to text, it facilitates subsequent summarization methods for a comprehensive and language-diverse analysis.

B. Data Preprocessing

Text Cleaning:
The initial preprocessing step involves the removal of non-Devanagari characters such as punctuation marks, numbers, and special symbols, as they may not contribute to the summarization process.

Tokenization:
Devanagari-specific tokenization techniques are employed to segment the text into individual tokens. This involves utilizing libraries capable of handling Unicode characters and applying word-level or subword-level tokenization to break down the text into meaningful units for further processing.

Stopword Removal:
A curated list of stopwords specific to Devanagari languages is created, consisting of common words that do not carry significant semantic meaning, such as articles and prepositions. These stopwords are then filtered out from the text to focus on content-bearing words during the summarization process.

Sentence Segmentation:
The text is segmented into individual sentences using appropriate boundary detection methods tailored to the Devanagari script.

C. Feature Extraction and Text Representation

PageRank-based Summarization:

Methodology: The PageRank algorithm, a web search engine strategy, uses graph theory to rank sentences based on their relevance to other sentences in the input text.

Output: An extractive summary is the result of PageRank-based summarization, in which sentences are chosen according to their graph ranking. Together, these chosen sentences highlight the most important details from the source material and give a succinct synopsis of the subject matter.

BART-based Summarization:

Methodology: The model uses BART (Bidirectional and Auto-Regressive Transformers) for bidirectional abstractive summarization, translating Devanagari text to English using pre-trained models like "ai4bharat/IndicBART," ensuring succinct and logical summaries by considering the text's subtleties and semantics.

Output: Abstractive summaries in devnagari are generated by BART-based summarizing. These summaries enable users to quickly and easily understand the primary concepts by maintaining the original text's key content while presenting it in a simplified and understandable format.

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:05:08 UTC from IEEE Xplore. Restrictions apply.

T5-based Summarization:

Methodology: The T5 (Text-to-Text Transfer Transformer) is utilized for multilingual summarization, offering flexibility in processing different input languages. It tokenizes Devanagari text and uses the model checkpoint "csebuetnlp/mT5_multilingual_XLSum," making multilingual summarization a smooth process due to its text-to-text structure.

Output: T5-based summarizing generates abstractive summaries that capture the main ideas of the incoming text in a manner like to BART. T5 can handle a variety of language situations thanks to its multilingual capabilities, which makes it appropriate for a broad range of summarization tasks.

LexRank-based Summarization:

Methodology: LexRank uses an extractive strategy, concentrating on locating the text's most significant sentences. The method creates an extractive summary by utilizing LexRank scores and semantic associations to rank sentences according to how similar they are to other sentences. This allows the system to keep important information while eliminating irrelevant or duplicate text.

Output: An extractive summary made up of sentences chosen according to their importance scores is the result of the LexRank-based summarizing process. LexRank guarantees that the summary preserves coherence and faithfulness to the original text by giving priority to sentences that are more pertinent to the overall context.

Comprehensive Analysis:

Approach: BART and T5 use transformer-based models for abstractive summarization, whereas PageRank and LexRank use graph-based techniques for extractive summarization.

Output Type: While BART and T5 create abstractive summaries that may rewrite and condense the information, PageRank and LexRank create extractive summaries by choosing sentences straight from the input text.

Dependency on Pre-training: While PageRank and LexRank algorithms do not require pre-training but may require fine-tuning for best performance, BART and T5 benefit from pre-trained models to complete summarization jobs efficiently.

Managing Multilingual Texts: T5 can handle input text in multiple languages thanks to its multilingual capabilities, which help it adapt to a variety of linguistic circumstances. For other models to handle multilingual texts efficiently, further preprocessing or certain setups could be needed.

Performance & Adaptability: Every model's efficacy is contingent upon various elements, including the features of the dataset, subtleties in language, and particular requirements for summarization. To evaluate their effectiveness and adaptation to various settings, a thorough assessment is necessary, using either human judgment or reference summaries.

IV. Result and Discussion:

The project utilized advanced summarization models, such as T5, BART, TextRank, and LexRank, to generate summarized text from the Devanagari dataset. The dataset contains 36,000 records, each containing titles, texts, and summaries. To evaluate the effectiveness of these models in real-world scenarios, 20 videos with a duration of less than 30 minutes were tested. The summaries were evaluated using ROUGE-1, ROUGE-2, and ROUGE-L metrics, considering precision, recall, and F-measure.

Rogue scores, particularly ROUGE-N and ROUGE-L, are commonly used in text summarization and natural language processing tasks to evaluate the quality of generated summaries compared to reference summaries or ground truth summaries. ROUGE-L measures the longest common subsequence (LCS) between the generated summary and reference summaries, while ROUGE-N measures the overlap of n-grams between the generated summary and reference summaries. These metrics are valuable for assessing the effectiveness of automatic summarization systems by quantifying how well the generated summary captures key information from reference summaries.

Precision measures the relevance of the generated summary compared to the reference summary, with high precision indicating mostly relevant information. *Recall* measures the completeness of the generated summary compared to the reference summary, with high recall indicating most relevant information from the reference summary. *F-measure* (F1-score) combines precision and recall into a single score, providing a balanced assessment of the summarization model, considering both relevance and completeness.

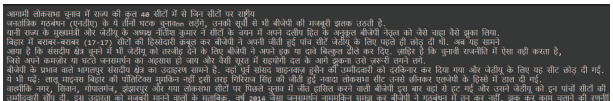

Fig 2: Input Paragraph

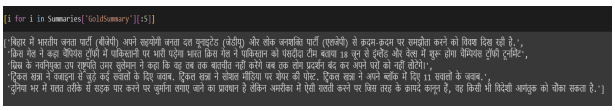

Fig 3: Reference Summary

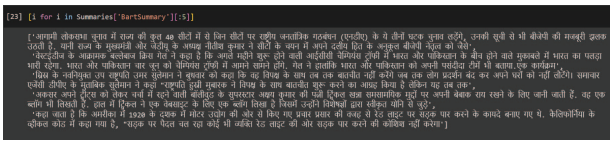

Fig 4: Bart Summary

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:05:08 UTC from IEEE Xplore. Restrictions apply.

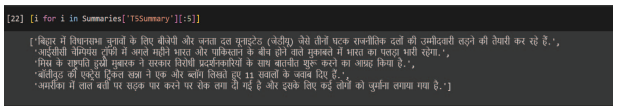

Fig 5: T5 Summary

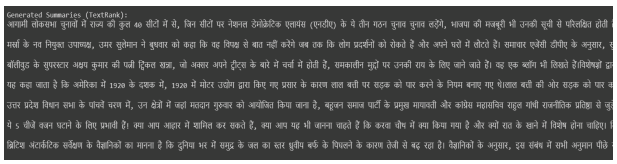

Fig 6: Textrank Summary (tf-idf)

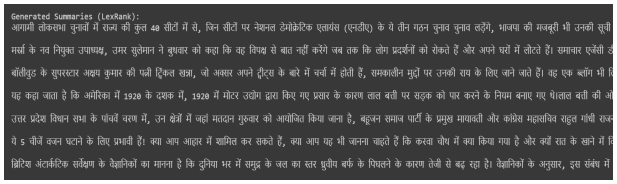

Fig 7: LexRank Summary

In the figures above, you can see the input paragraph in Fig 2 and corresponding Gold Summary or Reference summary in Fig 3. Next, we observe the outputs of various text summarization algorithms on the given input Paragraph in Fig 2, algorithms like BART, T5, TextRank, LexRank and their corresponding outputs in Fig 4, Fig 5, Fig 6 and Fig 7 respectively.

Techniques	Rouge-1			Rouge-2			Rouge-L		
Score	Precision	Recall	F-measure	Precision	Recall	F-measure	Precision	Recall	F-measure
Bart	0.3533	0.5887	0.4369	0.2045	0.3516	0.2570	0.3194	0.5320	0.3954
T5	0.3927	0.2991	0.3332	0.1470	0.1008	0.1184	0.3122	0.2383	0.2651
Textrank	0.1635	0.5798	0.2422	0.0589	0.2007	0.0869	0.1360	0.4765	0.2007
Lexrank	0.1195	0.6293	0.1939	0.0413	0.2404	0.0678	0.0955	0.5148	0.1557

Table I : Rough Score

We will explain the results shown in Table I for BART model summarisation, rest all can be understood in a similar manner.The Rouge-1 metric evaluates the overlap of unigrams (single words) between the generated summaries and reference summaries. With a recall of 0.5887, it indicates that, on average, the Bart model's summaries include approximately 58.88% of the unigrams found in the reference summaries. The precision, at 0.3533, suggests that around 35.33% of the unigrams in the Bart model's summaries are also present in the reference summaries. The F-measure, calculated as 0.4369, provides an overall assessment, balancing both precision and recall for unigram overlap.

For Rouge-2, the metric assesses the overlap of bigrams (pairs of words) between the generated and reference summaries. The recall of 0.3516 implies that, on average, Bart's summaries contain about 35.16% of the bigrams present in the reference summaries. With a precision of 0.2045, approximately 20.45% of the bigrams in Bart's summaries match those in the reference summaries. The F-measure, at 0.2570, offers a balanced assessment of bigram overlap. Lastly, Rouge-L evaluates the longest common subsequence (LCS) between the generated and reference summaries. With a recall of 0.532056, it suggests that, on average, Bart's summaries include about 53.21% of the longest common subsequences present in the reference summaries. The precision, at 0.319403, indicates that around 31.94% of the LCS in Bart's summaries match those in the reference summaries. The F-measure, calculated as 0.395411, provides an overall assessment, balancing both precision and recall for LCS overlap. Overall the BART model for text summarization gave the most useful results with a largest F-measure of all the ROGUE-1, ROGUE-2 and ROGUE-L metrics.

V. Conclusion and Future Scope

The implementation of our project, aimed at automating the conversion of YouTube videos into concise Devanagari text summaries, has showcased significant advancements through the integration of state-of-the-art natural language processing (NLP) techniques and summarization models.The utilization of diverse summarization models allowed for a comprehensive exploration of both generative (T5, BART) and extractive (TextRank, LexRank) approaches. This diversity facilitated a nuanced understanding of the trade-offs and benefits associated with each methodology.T5 and BART demonstrated remarkable linguistic understanding, producing summaries with high accuracy and coherence.Traditional extractive models, TextRank and LexRank, showcased efficiency and brevity in their summaries. Length of video should be less than or equal to 30 minutes. In the future, we intend to make a chrome extension for summarizing YouTube videos. The extension can fetch the YouTube video's URL from the Search bar and show the summarized video text on a different page.

VI. References

[1] N. Kapoor, A. Gupta and K. Meenakshi, "Automatic Extractive Text Summarization of Hindi Text using Deep learning approach," 2023 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 2023

[2] S. Degadwala, D. Vyas, K. N. Patel, M. Soni, P. Parkash Singh and R. Maranan, "Optimizing Hindi Paragraph Summarization through PageRank Method," 2023 2nd International Conference on Edge Computing and Applications (ICECAA), Namakkal, India, 2023

[3] S. Sood, M. F. Ukrit and R. Parasar, "Analysis of Extractive Text Summarisation Techniques for Multilingual Texts," 2023 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI), Chennai, India, 2023

[4] R. Karmakar, K. Nirantar, P. Kurunkar, P. Hiremath and D. Chaudhari, "Indian Regional Language Abstractive Text Summarization using Attention-based LSTM Neural Network," 2021 International Conference on Intelligent Technologies (CONIT), Hubli, India, 2021

[5] V. Pratap et al., "Wav2Letter++: A Fast Open-source Speech Recognition System," ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Brighton, UK, 2019, pp. 6460-6464, doi: 10.1109/ICASSP.2019.8683535.

[6] Lucas Rafael Stefanel Gris, Ricardo Marcacini*, Arnaldo andido Junior, Edresson Casanova, Anderson Soares, and Sandra Maria Aluísio,"Evaluating OpenAI's Whisper ASR for Punctuation Prediction and Topic Modeling of life histories of the Museum of the Person", DE gruyter, 2023

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:05:08 UTC from IEEE Xplore. Restrictions apply.

[7] Gupta, A., Chugh, D., Anjum, Katarya, R. (2022). Automated News Summarization Using Transformers. In: Aurelia, S., Hiremath, S.S., Subramanian, K., Biswas, S.K. (eds) Sustainable Advanced Computing. Lecture Notes in Electrical Engineering, vol 840. Springer,Singapore.

[8] A. Shah, D. Zanzmera, and K. Mehta, ”Deep Learning based Automatic Hindi Text Summarization,” in 2022 6th International Conference on Computing Methodologies and Communication (ICCMC) 2022, 10.1109/ICCMC53470.2022.9753735. 1455-1461. doi

[9] S. Deo and D. Banik, "Text Summarization using Textrank and Lexrank through Latent Semantic analysis," 2022 OITS International Conference on Information Technology (OCIT), Bhubaneswar,India,2022, pp. 113-118, doi: 10.1109/OCIT56763.2022.00031.

[10] B. N, D. Kumari, B. N, M. N, S. K. P and S. R. A, "Text Summarization using NLP Technique," 2022 International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER), Shivamogga, India, 2022, pp. 30-35, doi: 10.1109/DISCOVER55800.2022.9974823.

[11] C. Kwatra and K. Gupta, "Extractive and Abstractive Summarization for Hindi Text using Hierarchical Clustering." in 2021 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES), 2021, pp. 1-6. doi: 10.1109/ICSES52305.2021.9633789.

[12] K. Tawatia, N. Jain and S. Kundu, "Hindi Document Extractive Summarization: Neural Method on A New Data Set," *2022 5th International Conference on Computational Intelligence and Networks (CINE)*, Bhubaneswar, India, 2022, pp. 1-6, doi: 10.1109/CINE56307.2022.10037327.

[13] N. Moratanch and S. Chitrakala, "A survey on abstractive text summarization," 2016 International Conference on Circuit, Power and Computing Technologies (ICCPCT), Nagercoil, India, 2016, pp. 1-7, doi: 10.1109/ICCPCT.2016.7530193.

[14] A. A. Syed, F. L. Gaol and T. Matsuo, "A Survey of the State-of-the-Art Models in Neural Abstractive Text Summarization," in IEEE Access, vol. 9, pp. 13248-13265, 2021, doi: 10.1109/ACCESS.2021.3052783.

[15] Akanksha Joshi, E. Fidalgo, E. Alegre, Laura Fernández-Robles, SummCoder: An unsupervised framework for extractive text summarization based on deep auto-encoders, Expert Systems with Applications, Volume 129, 2019, Pages 200-215, ISSN 0957-4174.

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:05:08 UTC from IEEE Xplore. Restrictions apply.